

Image Processing: 2D Fast Fourier Transform

Milestone 2 — RISC-V Assembly Implementation

Group Member: Narin Nadeem

GitHub: https://github.com/narinnadeem/2D_FFT_Image_Processing/tree/milestone-2

Team Name: Core

Contents

1	Math Helper Functions	2
1.1	my_sin — Sine Approximation	2
1.2	my_cos — Cosine Approximation	3
1.3	log2_int — Integer Log Base 2	3
2	Twiddle Factor Generation	5
3	Bit Reversal	6
4	Array Reordering	8
5	Recursive FFT	10
5.1	fft_1d_recursive — Wrapper Function	11
6	Iterative FFT	15
6.1	fft_1d_iterative — Wrapper Function	16
7	Performance Comparison — Green Computing	18
7.1	Measurements	18
7.2	Analysis	18
8	Challenges and Solutions	20
8.1	RISC-V Floating Point Calling Convention	20
8.2	Twiddle Factor Indexing in Recursive FFT	20
8.3	GitHub Authentication	20
8.4	Milestone 1 Edge Detection Bug	20

1 Math Helper Functions

Since RISC-V assembly has no access to standard math libraries, all trigonometric functions were implemented from scratch using Taylor series approximations. These functions are the foundation of the FFT, they compute the twiddle factors (complex exponentials) used at each butterfly stage.

1.1 my_sin — Sine Approximation

The sine function uses a 10-term Taylor series:

$$\sin(x) = x - \frac{x^3}{3!} + \frac{x^5}{5!} - \frac{x^7}{7!} + \dots$$

Before applying the series, the input is reduced to $[-\pi, \pi]$ by repeatedly adding or subtracting 2π (≈ 6.283). This is critical because Taylor series converge poorly far from zero.

```

1      .globl my_sin
2      .type  my_sin, @function
3 my_sin:
4      addi    sp, sp, -32
5      sd      ra, 24(sp)
6      fsd     fs0, 16(sp)
7      fsd     fs1, 8(sp)
8      fsd     fs2, 0(sp)
9      fmv.s   fs0, fa0           # fs0 = x (working copy)
10
11     # Argument reduction: wrap x to [-PI, PI]
12     la       t0, TWO_PI
13     flw      ft1, 0(t0)
14     la       t0, PI
15     flw      ft2, 0(t0)        # ft2 = PI
16 sin_reduce_pos:
17     flt.s    t0, ft2, fs0      # if PI < x
18     beqz     t0, sin_reduce_neg
19     fsub.s    fs0, fs0, ft1     # x -= 2*PI
20     j        sin_reduce_pos
21 sin_reduce_neg:
22     fneg.s    ft3, ft2         # ft3 = -PI
23     flt.s     t0, fs0, ft3     # if x < -PI
24     beqz     t0, sin_taylor
25     fadd.s    fs0, fs0, ft1     # x += 2*PI
26     j        sin_reduce_neg
27
28     # Taylor series (10 terms)
29 sin_taylor:
30     fmv.w.x   fs1, zero        # res = 0.0
31     fmv.s     fs2, fs0         # term = x
32     fmul.s     ft4, fs0, fs0    # x_sq = x*x
33     li        t1, 1           # i = 1
34 sin_loop:
35     li        t2, 11
36     bge       t1, t2, sin_done
37     fadd.s     fs1, fs1, fs2    # res += term
38     slli      t3, t1, 1        # 2i

```

```

39     addi    t4, t3, 1           # 2i+1
40     mul     t5, t3, t4         # (2i)*(2i+1)
41     fcvt.s.w ft5, t5
42     fdiv.s  ft6, ft4, ft5      # x_sq / denom
43     fneg.s  ft6, ft6
44     fmul.s  fs2, fs2, ft6      # term *= (-x_sq/denom)
45     addi    t1, t1, 1
46     j       sin_loop
47 sin_done:
48     fmv.s   fa0, fs1           # return res
49     fld     fs2, 0(sp)
50     fld     fs1, 8(sp)
51     fld     fs0, 16(sp)
52     ld      ra, 24(sp)
53     addi    sp, sp, 32
54     ret

```

Listing 1: my_sin: Taylor series sine in RISC-V assembly

1.2 my_cos — Cosine Approximation

The cosine exploits the identity $\cos(x) = \sin(x + \pi/2)$, implemented as a tail-call to `my_sin` after shifting the argument by $\pi/2$. This avoids duplicating the Taylor series logic entirely. Note: even if $x = \pi$, then $x + \pi/2 = 1.5\pi$, which is outside $[-\pi, \pi]$. The argument reduction loop inside `my_sin` correctly handles this by subtracting 2π to bring $1.5\pi \rightarrow -0.5\pi$ before computing the series.

```

1     .globl  my_cos
2     .type   my_cos, @function
3 my_cos:
4     la      t0, HALF_PI
5     flw     ft0, 0(t0)
6     fadd.s  fa0, fa0, ft0      # x += PI/2
7     j       my_sin           # tail-call my_sin

```

Listing 2: my_cos: tail-call to my_sin

1.3 log2_int — Integer Log Base 2

This helper computes $\lfloor \log_2(n) \rfloor$ by repeatedly right-shifting n until it reaches 1. It is used by `bit_reverse_array` to determine the number of bits required for index reversal.

```

1     .globl  log2_int
2     .type   log2_int, @function
3 log2_int:
4     li      a1, 0
5 l2_loop:
6     li      t0, 1
7     ble     a0, t0, l2_done
8     srli    a0, a0, 1          # n >>= 1
9     addi    a1, a1, 1          # count++
10    j       l2_loop
11 l2_done:
12    mv      a0, a1
13    ret

```

Listing 3: log2_int: integer log base 2

Validation: The terminal output below shows sin and cos assembly results matching the C `sinf/cosf` reference values exactly for all test angles.

```

=====
Milestone 2: 1D FFT in RISC-V Assembly
=====
=== Math Function Validation ===
angle=0      sin: asm= 0.0000 ref= 0.0000 | cos: asm= 1.0000 ref= 1.0000
angle=PI/4   sin: asm= 0.7071 ref= 0.7071 | cos: asm= 0.7071 ref= 0.7071
angle=PI/2   sin: asm= 1.0000 ref= 1.0000 | cos: asm=-0.0000 ref=-0.0000
angle=PI     sin: asm=-0.0000 ref=-0.0000 | cos: asm=-1.0000 ref=-1.0000
angle=-PI/2  sin: asm=-1.0000 ref=-1.0000 | cos: asm=-0.0000 ref=-0.0000

=== Bit Reversal Validation (n=8, log_n=3) ===
  Index | Binary | Bit-Rev Binary | Bit-Rev Index | Assembly
  -----|-----|-----|-----|-----
    0 | 000 | 000 | 0 | 0 OK
    1 | 001 | 004 | 4 | 4 OK
    2 | 002 | 002 | 2 | 2 OK
    3 | 003 | 006 | 6 | 6 OK
    4 | 004 | 001 | 1 | 1 OK
    5 | 005 | 005 | 5 | 5 OK
    6 | 006 | 003 | 3 | 3 OK
    7 | 007 | 007 | 7 | 7 OK

=== Array Reorder Validation (n=8) ===
Before: 0 1 2 3 4 5 6 7
After:  0 4 2 6 1 5 3 7
Expected: 0 4 2 6 1 5 3 7

```

Figure 1: Math function validation, bit reversal, and array reorder terminal output

2 Twiddle Factor Generation

Twiddle factors are the complex exponentials:

$$W_N^k = e^{-j2\pi k/N} = \cos\left(\frac{-2\pi k}{N}\right) + j \sin\left(\frac{-2\pi k}{N}\right)$$

For a forward FFT of size N , we need $N/2$ twiddle factors ($k = 0$ to $N/2 - 1$). The real and imaginary parts are stored in two separate float arrays allocated on the heap by the C driver.

The key insight is the **SIMD** (Single Instruction, Multiple Data) principle: due to the periodicity and symmetry of complex exponentials, all butterfly stages can reuse the same $N/2$ precomputed values by stepping through the array with a stride that halves each stage.

```

1  .globl generate_twiddle_factors
2  .type generate_twiddle_factors, @function
3  generate_twiddle_factors:
4      addi    sp, sp, -64
5      sd      ra, 56(sp)
6      # s0=twiddle_real ptr, s1=twiddle_imag ptr, s2=n, s3=k
7      mv      s0, a0
8      mv      s1, a1
9      mv      s2, a2
10     li      s3, 0                # k = 0
11     la      t1, NEG_TWO_PI
12     flw     fs0, 0(t1)          # fs0 = -2*PI
13 twiddle_loop:
14     srli    t0, s2, 1
15     bge     s3, t0, twiddle_done # k >= n/2 -> stop
16
17     # angle = -2*PI*k / n
18     fcvt.s.w fa0, s3
19     fmul.s  fa0, fs0, fa0        # -2*PI*k
20     fcvt.s.w ft2, s2
21     fdiv.s  fa0, fa0, ft2        # angle = -2*PI*k/n
22
23     fsd     fa0, 0(sp)          # spill angle before call
24     call    my_cos
25     fsw     fa0, 0(s0)          # twiddle_real[k] = cos(angle)
26     addi    s0, s0, 4
27
28     flw     fa0, 0(sp)          # reload angle
29     call    my_sin
30     fsw     fa0, 0(s1)          # twiddle_imag[k] = sin(angle)
31     addi    s1, s1, 4
32
33     addi    s3, s3, 1           # k++
34     j       twiddle_loop
35 twiddle_done:
36     ld      ra, 56(sp)
37     addi    sp, sp, 64
38     ret

```

Listing 4: generate_twiddle_factors: precompute W_N^k for $k=0..N/2-1$

3 Bit Reversal

The Cooley-Tukey Decimation-in-Time (DIT) FFT requires input samples to be in bit-reversed order before the butterfly stages begin. The `reverse_bits` function extracts the LSB of x on each iteration, shifts it into the result in reverse order, and returns the reversed value.

```

1  .globl reverse_bits
2  .type  reverse_bits, @function
3  reverse_bits:
4      # a0 = x, a1 = log_n, returns reversed in a0
5      li    t0, 0                # reversed = 0
6      mv    t1, a1                # counter = log_n
7  rb_loop:
8      beqz   t1, rb_done
9      slli   t0, t0, 1            # reversed <= 1
10     andi   t2, a0, 1            # t2 = x & 1 (LSB)
11     or     t0, t0, t2           # reversed |= LSB
12     srli   a0, a0, 1            # x >>= 1
13     addi   t1, t1, -1
14     j      rb_loop
15  rb_done:
16     mv     a0, t0
17     ret

```

Listing 5: `reverse_bits`: reverse the low `log_n` bits of `x`

Screenshot of terminal output showing correctness:

```

=====
Milestone 2: 1D FFT in RISC-V Assembly
=====
=== Math Function Validation ===
angle=0      sin: asm= 0.0000 ref= 0.0000 | cos: asm= 1.0000 ref= 1.0000
angle=PI/4   sin: asm= 0.7071 ref= 0.7071 | cos: asm= 0.7071 ref= 0.7071
angle=PI/2   sin: asm= 1.0000 ref= 1.0000 | cos: asm=-0.0000 ref=-0.0000
angle=PI     sin: asm=-0.0000 ref=-0.0000 | cos: asm=-1.0000 ref=-1.0000
angle=-PI/2  sin: asm=-1.0000 ref=-1.0000 | cos: asm=-0.0000 ref=-0.0000

=== Bit Reversal Validation (n=8, log_n=3) ===
  Index | Binary | Bit-Rev Binary | Bit-Rev Index | Assembly
  -----|-----|-----|-----|-----
    0 | 000 | 000 | 0 | 0 OK
    1 | 001 | 004 | 4 | 4 OK
    2 | 002 | 002 | 2 | 2 OK
    3 | 003 | 006 | 6 | 6 OK
    4 | 004 | 001 | 1 | 1 OK
    5 | 005 | 005 | 5 | 5 OK
    6 | 006 | 003 | 3 | 3 OK
    7 | 007 | 007 | 7 | 7 OK

=== Array Reorder Validation (n=8) ===
Before: 0 1 2 3 4 5 6 7
After:  0 4 2 6 1 5 3 7
Expected: 0 4 2 6 1 5 3 7

```

Figure 2: Bit reversal validation for $n=8$: all 8 indices show “OK”

4 Array Reordering

`bit_reverse_array` iterates over indices 0 to $n - 1$, computes the bit-reversed partner j for each i , and swaps elements at positions i and j only when $i < j$ (to avoid double-swapping). Both real and imaginary arrays are reordered identically.

```

1  .globl bit_reverse_array
2  .type bit_reverse_array, @function
3  bit_reverse_array:
4  addi    sp, sp, -48
5  sd      ra, 40(sp)
6  # s0=xr, s1=xi, s2=n, s3=i, s4=log_n
7  mv      s0, a0
8  mv      s1, a1
9  mv      s2, a2
10 li      s3, 0                # i = 0
11
12 mv      a0, s2
13 call    log2_int             # log_n = log2_int(n)
14 mv      s4, a0
15
16 bra_loop:
17 bge     s3, s2, bra_done     # i >= n -> stop
18
19 mv      a0, s3
20 mv      a1, s4
21 call    reverse_bits         # j = reverse_bits(i, log_n)
22 mv      t0, a0               # t0 = j
23
24 ble     t0, s3, bra_next     # only swap if i < j
25
26 # swap xr[i] <-> xr[j]
27 slli    t1, s3, 2
28 add     t1, s0, t1
29 slli    t2, t0, 2
30 add     t2, s0, t2
31 flw     ft0, 0(t1)
32 flw     ft1, 0(t2)
33 fsw     ft1, 0(t1)
34 fsw     ft0, 0(t2)
35 # same swap for xi[i] <-> xi[j]
36 slli    t1, s3, 2
37 add     t1, s1, t1
38 slli    t2, t0, 2
39 add     t2, s1, t2
40 flw     ft0, 0(t1)
41 flw     ft1, 0(t2)
42 fsw     ft1, 0(t1)
43 fsw     ft0, 0(t2)
44 bra_next:
45 addi    s3, s3, 1
46 j       bra_loop
47 bra_done:
48 ld      ra, 40(sp)
49 addi    sp, sp, 48
50 ret

```

Listing 6: `bit_reverse_array`: reorder input using bit-reversed indices

Screenshot of terminal output showing correctness:

```

=====
Milestone 2: 1D FFT in RISC-V Assembly
=====
=== Math Function Validation ===
angle=0      sin: asm= 0.0000 ref= 0.0000 | cos: asm= 1.0000 ref= 1.0000
angle=PI/4   sin: asm= 0.7071 ref= 0.7071 | cos: asm= 0.7071 ref= 0.7071
angle=PI/2   sin: asm= 1.0000 ref= 1.0000 | cos: asm=-0.0000 ref=-0.0000
angle=PI     sin: asm=-0.0000 ref=-0.0000 | cos: asm=-1.0000 ref=-1.0000
angle=-PI/2  sin: asm=-1.0000 ref=-1.0000 | cos: asm=-0.0000 ref=-0.0000

=== Bit Reversal Validation (n=8, log_n=3) ===
Index | Binary | Bit-Rev Binary | Bit-Rev Index | Assembly
-----|-----|-----|-----|-----
0 | 000 | 000 | 0 | 0 OK
1 | 001 | 004 | 4 | 4 OK
2 | 002 | 002 | 2 | 2 OK
3 | 003 | 006 | 6 | 6 OK
4 | 004 | 001 | 1 | 1 OK
5 | 005 | 005 | 5 | 5 OK
6 | 006 | 003 | 3 | 3 OK
7 | 007 | 007 | 7 | 7 OK

=== Array Reorder Validation (n=8) ===
Before: 0 1 2 3 4 5 6 7
After:  0 4 2 6 1 5 3 7
Expected: 0 4 2 6 1 5 3 7

```

Figure 3: Array reorder validation: Before: 0 1 2 3 4 5 6 7 → After: 0 4 2 6 1 5 3 7 (matches expected)

5 Recursive FFT

`butterfly_recursive` implements the Cooley-Tukey algorithm recursively. Given an array of size n , it recursively processes the first half, then the second half, then combines them using the butterfly operation. The base case is $n \leq 1$. The twiddle index for position j at a sub-problem of size n is:

$$\text{twiddle_idx} = j \times \frac{N_{\text{total}}}{n}$$

```

1  .globl butterfly_recursive
2  .type butterfly_recursive, @function
3  butterfly_recursive:
4      # a0=xr, a1=xi, a2=tr, a3=ti, a4=n, a5=total_n
5      li      t0, 1
6      ble     a4, t0, br_base      # base case: n <= 1
7      addi    sp, sp, -64
8      sd      ra, 56(sp)
9      # save s0-s6 ...
10     mv      s0, a0 # xr
11     mv      s1, a1 # xi
12     mv      s2, a2 # tr
13     mv      s3, a3 # ti
14     mv      s4, a4 # n
15     mv      s5, a5 # total_n
16     srli     s6, s4, 1           # half = n/2
17
18     # Recurse on first half
19     mv      a0, s0
20     mv      a1, s1
21     mv      a2, s2
22     mv      a3, s3
23     mv      a4, s6
24     mv      a5, s5
25     call     butterfly_recursive
26
27     # Recurse on second half (xr+half, xi+half)
28     slli     t0, s6, 2
29     add      a0, s0, t0
30     add      a1, s1, t0
31     mv      a2, s2
32     mv      a3, s3
33     mv      a4, s6
34     mv      a5, s5
35     call     butterfly_recursive
36
37     # Combine: butterfly for j = 0 to half-1
38     div      t2, s5, s4          # step = total_n / n
39     li      t3, 0               # j = 0
40 br_combine:
41     bge      t3, s6, br_combine_done
42     mul      t4, t3, t2          # twiddle_idx = j * step
43     slli     t5, t4, 2
44     add      t5, s2, t5
45     flw      ft0, 0(t5)         # wr = tr[twiddle_idx]
46     slli     t5, t4, 2
47     add      t5, s3, t5

```

```

48     flw      ft1, 0(t5)           # wi = ti[twiddle_idx]
49     add      t5, t3, s6          # t5 = j + half
50     slli     t6, t5, 2
51     add      t6, s0, t6
52     flw      ft2, 0(t6)          # xr[j+half]
53     slli     t6, t5, 2
54     add      t6, s1, t6
55     flw      ft3, 0(t6)          # xi[j+half]
56     fmul.s   ft4, ft2, ft0
57     fmul.s   ft5, ft3, ft1
58     fsub.s   ft4, ft4, ft5       # tr_val = xr[j+half]*wr - xi[j+half]
                                   # *wi
59     fmul.s   ft5, ft2, ft1
60     fmul.s   ft6, ft3, ft0
61     fadd.s   ft5, ft5, ft6       # ti_val = xr[j+half]*wi + xi[j+half]
                                   # *wr
62     slli     t6, t3, 2
63     add      t6, s0, t6
64     flw      ft6, 0(t6)          # xr[j]
65     slli     t6, t3, 2
66     add      t6, s1, t6
67     flw      ft7, 0(t6)          # xi[j]
68     fsub.s   fa5, ft6, ft4       # xr[j+half] = xr[j] - tr_val
69     add      t6, t3, s6
70     slli     t6, t6, 2
71     add      t6, s0, t6
72     fsw      fa5, 0(t6)
73     fsub.s   fa5, ft7, ft5       # xi[j+half] = xi[j] - ti_val
74     add      t6, t3, s6
75     slli     t6, t6, 2
76     add      t6, s1, t6
77     fsw      fa5, 0(t6)
78     fadd.s   fa5, ft6, ft4       # xr[j] = xr[j] + tr_val
79     slli     t6, t3, 2
80     add      t6, s0, t6
81     fsw      fa5, 0(t6)
82     fadd.s   fa5, ft7, ft5       # xi[j] = xi[j] + ti_val
83     slli     t6, t3, 2
84     add      t6, s1, t6
85     fsw      fa5, 0(t6)
86     addi     t3, t3, 1
87     j        br_combine
88 br_combine_done:
89     ld       ra, 56(sp)
90     addi     sp, sp, 64
91     ret
92 br_base:
93     ret

```

Listing 7: butterfly_recursive: recursive Cooley-Tukey butterfly

5.1 fft_1d_recursive — Wrapper Function

```

1     .globl  fft_1d_recursive
2     .type   fft_1d_recursive, @function
3 fft_1d_recursive:
4     addi    sp, sp, -48

```

```
5      sd      ra, 40(sp)
6      # Save a0-a4 into s0-s4
7      mv      s0, a0
8      mv      s1, a1
9      mv      s2, a2
10     mv      s3, a3
11     mv      s4, a4
12     # Step 1: bit-reverse the input
13     mv      a0, s0
14     mv      a1, s1
15     mv      a2, s4
16     call     bit_reverse_array
17     # Step 2: recursive butterfly with total_n = n
18     mv      a0, s0
19     mv      a1, s1
20     mv      a2, s2
21     mv      a3, s3
22     mv      a4, s4
23     mv      a5, s4          # total_n = n
24     call     butterfly_recursive
25     ld      ra, 40(sp)
26     addi     sp, sp, 48
27     ret
```

Listing 8: fft_1d_recursive: wrapper calling bit_reverse then butterfly

Validation: RISC-V assembly output cross-checked against FFT Calculator Online

The website <https://calculatorshub.net/mathematical-calculators/fft-calculator/> was used to verify correctness. Input: all-ones sequence of length 8.

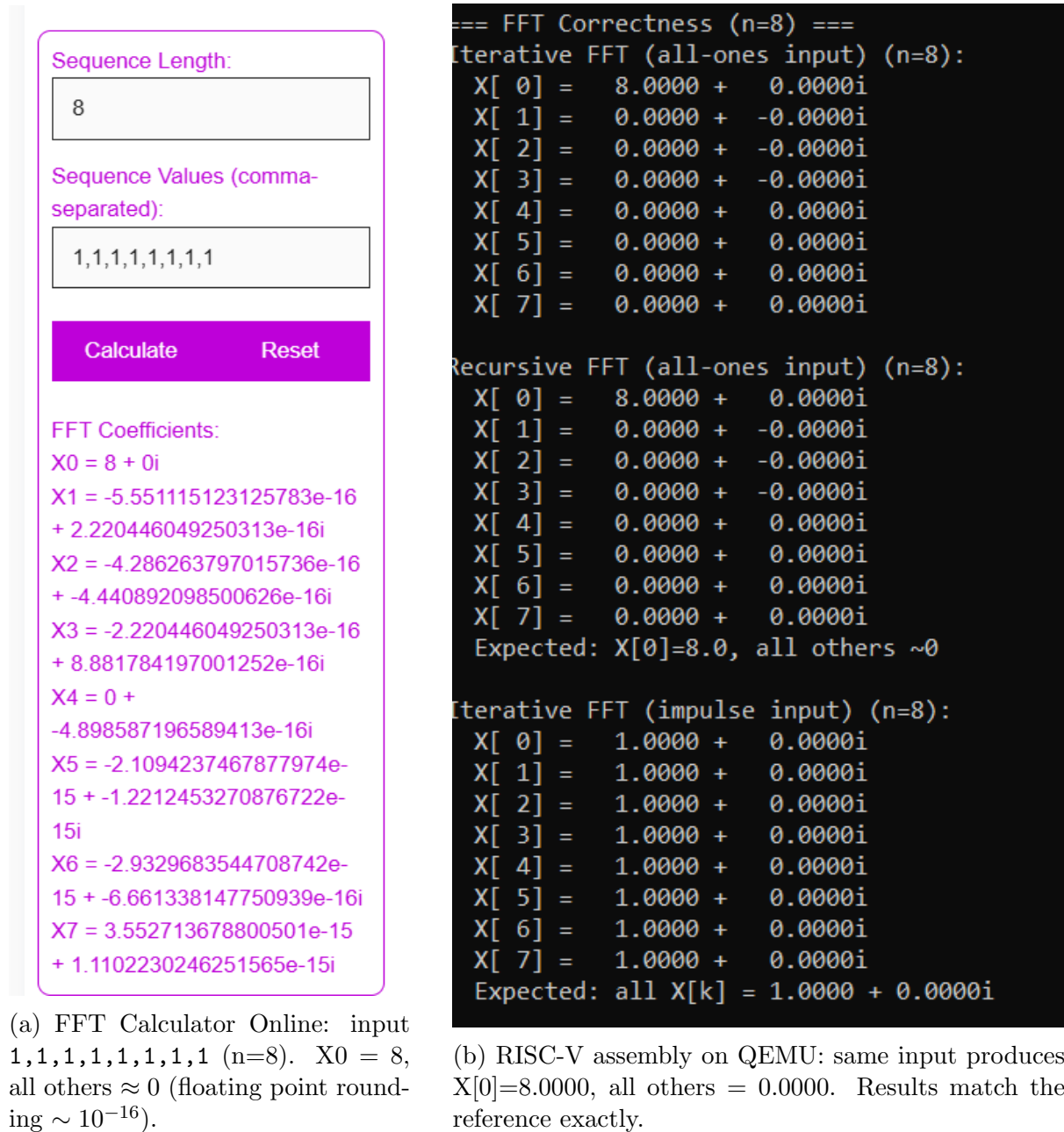


Figure 4: Cross-validation: FFT Calculator Online (left) vs RISC-V assembly on QEMU (right). Both produce $X[0]=8$ and all other coefficients ≈ 0 for all-ones input.

```

narin_nadeem@DESKTOP-E5UU0GK:~/milestone2$ python3 -c "
import cmath, math
x = [1,1,1,1,1,1,1,1]
n = len(x)
X = [sum(x[k]*cmath.exp(-2j*math.pi*k*i/n) for k in range(n)) for i in range(n)]
print('FFT of all-ones (n=8):')
for i,v in enumerate(X):
    print(f' X[{i}] = {v.real:.4f} + {v.imag:.4f}i')
"
FFT of all-ones (n=8):
X[0] = 8.0000 + 0.0000i
X[1] = -0.0000 + 0.0000i
X[2] = -0.0000 + -0.0000i
X[3] = -0.0000 + 0.0000i
X[4] = 0.0000 + -0.0000i
X[5] = -0.0000 + -0.0000i
X[6] = -0.0000 + -0.0000i
X[7] = 0.0000 + 0.0000i

```

(a) Additional cross-check: Python DFT for all-ones ($n=8$). $X[0]=8$, rest ≈ 0 .

```

narin_nadeem@DESKTOP-E5UU0GK:~/milestone2$ python3 -c "
import cmath, math
x = [1,0,0,0,0,0,0,0]
n = len(x)
X = [sum(x[k]*cmath.exp(-2j*math.pi*k*i/n) for k in range(n)) for i in range(n)]
print('FFT of impulse (n=8):')
for i,v in enumerate(X):
    print(f' X[{i}] = {v.real:.4f} + {v.imag:.4f}i')
"
FFT of impulse (n=8):
X[0] = 1.0000 + 0.0000i
X[1] = 1.0000 + 0.0000i
X[2] = 1.0000 + 0.0000i
X[3] = 1.0000 + 0.0000i
X[4] = 1.0000 + 0.0000i
X[5] = 1.0000 + 0.0000i
X[6] = 1.0000 + 0.0000i
X[7] = 1.0000 + 0.0000i

```

(b) Additional cross-check: Python DFT for impulse ($n=8$). All $X[k]=1$.

Figure 5: Further validation using Python DFT, all results match assembly output.

6 Iterative FFT

`butterfly_iterative` implements the same algorithm using nested loops over $\log_2(n)$ stages. In stage s , the butterfly group size is $m = 2^s$. Within each group, pairs of elements are combined using the twiddle factor at index $j \times (n/m)$. This avoids the overhead of recursive function calls.

```

1  .globl butterfly_iterative
2  .type butterfly_iterative, @function
3  butterfly_iterative:
4      # a0=xr, a1=xi, a2=tr, a3=ti, a4=n
5      addi    sp, sp, -80
6      sd      ra, 72(sp)
7      # save s0-s8 ...
8      mv      s4, a4                # n
9      mv      a0, s4
10     call    log2_int
11     mv      s5, a0                # stages = log2(n)
12     li      s6, 1                 # stage = 1
13 bi_stage_loop:
14     bgt      s6, s5, bi_done
15     li      t0, 1
16     sll      t0, t0, s6            # m = 1 << stage
17     srli     t1, t0, 1             # half_m = m/2
18     div      t2, s4, t0            # twiddle_step = n/m
19     li      s7, 0                 # k = 0
20 bi_k_loop:
21     bge      s7, s4, bi_stage_next
22     li      s8, 0                 # j = 0
23 bi_j_loop:
24     bge      s8, t1, bi_k_next
25     # top = k+j, bottom = k+j+half_m
26     add      t3, s7, s8
27     add      t4, t3, t1
28     # twiddle_idx = j * twiddle_step
29     mul      t5, s8, t2
30     # load wr=tr[t5], wi=ti[t5]
31     # load xr[top], xi[top], xr[bot], xi[bot]
32     # tr_val = wr*xr[bot] - wi*xi[bot]
33     # ti_val = wr*xi[bot] + wi*xr[bot]
34     # xr[bot] = xr[top] - tr_val
35     # xi[bot] = xi[top] - ti_val
36     # xr[top] = xr[top] + tr_val
37     # xi[top] = xi[top] + ti_val
38     addi     s8, s8, 1
39     j        bi_j_loop
40 bi_k_next:
41     add      s7, s7, t0            # k += m
42     j        bi_k_loop
43 bi_stage_next:
44     addi     s6, s6, 1             # stage++
45     j        bi_stage_loop
46 bi_done:
47     ld      ra, 72(sp)
48     addi     sp, sp, 80
49     ret

```

Listing 9: `butterfly_iterative`: iterative Cooley-Tukey butterfly

6.1 fft_1d_iterative — Wrapper Function

```
1  .globl fft_1d_iterative
2  .type  fft_1d_iterative, @function
3  fft_1d_iterative:
4      addi    sp, sp, -48
5      sd      ra, 40(sp)
6      mv      s0, a0
7      mv      s1, a1
8      mv      s2, a2
9      mv      s3, a3
10     mv      s4, a4
11     # Step 1: bit-reverse the input
12     mv      a0, s0
13     mv      a1, s1
14     mv      a2, s4
15     call     bit_reverse_array
16     # Step 2: iterative butterfly
17     mv      a0, s0
18     mv      a1, s1
19     mv      a2, s2
20     mv      a3, s3
21     mv      a4, s4
22     call     butterfly_iterative
23     ld      ra, 40(sp)
24     addi    sp, sp, 48
25     ret
```

Listing 10: fft_1d_iterative: wrapper calling bit_reverse then iterative butterfly

Screenshot of correct iterative and recursive FFT output on QEMU:

```

=== FFT Correctness (n=8) ===
Iterative FFT (all-ones input) (n=8):
  X[ 0] =  8.0000 +  0.0000i
  X[ 1] =  0.0000 + -0.0000i
  X[ 2] =  0.0000 + -0.0000i
  X[ 3] =  0.0000 + -0.0000i
  X[ 4] =  0.0000 +  0.0000i
  X[ 5] =  0.0000 +  0.0000i
  X[ 6] =  0.0000 +  0.0000i
  X[ 7] =  0.0000 +  0.0000i

Recursive FFT (all-ones input) (n=8):
  X[ 0] =  8.0000 +  0.0000i
  X[ 1] =  0.0000 + -0.0000i
  X[ 2] =  0.0000 + -0.0000i
  X[ 3] =  0.0000 + -0.0000i
  X[ 4] =  0.0000 +  0.0000i
  X[ 5] =  0.0000 +  0.0000i
  X[ 6] =  0.0000 +  0.0000i
  X[ 7] =  0.0000 +  0.0000i
  Expected: X[0]=8.0, all others ~0

Iterative FFT (impulse input) (n=8):
  X[ 0] =  1.0000 +  0.0000i
  X[ 1] =  1.0000 +  0.0000i
  X[ 2] =  1.0000 +  0.0000i
  X[ 3] =  1.0000 +  0.0000i
  X[ 4] =  1.0000 +  0.0000i
  X[ 5] =  1.0000 +  0.0000i
  X[ 6] =  1.0000 +  0.0000i
  X[ 7] =  1.0000 +  0.0000i
  Expected: all X[k] = 1.0000 + 0.0000i

```

Figure 6: QEMU terminal: both iterative and recursive FFT produce $X[0]=8$ for all-ones input and all $X[k]=1$ for impulse input ($n=8$), matching the DFT reference exactly.

7 Performance Comparison — Green Computing

Both FFT implementations were benchmarked on QEMU using the RISC-V `rdcycle` instruction to read the CPU cycle counter before and after each FFT call. Three input sizes were tested: 8, 16, and 32 samples.

7.1 Measurements

```

=== Performance Benchmarks ===

+-----+-----+-----+-----+-----+
| N      | Iter Cycles | Rec Cycles | Iter Stack(B) | Rec Stack(B) |
+-----+-----+-----+-----+-----+
| 8      | 52327       | 59034      | 176           | 288          |
| 16     | 8316        | 8861       | 176           | 352          |
| 32     | 14608       | 16845      | 176           | 416          |
+-----+-----+-----+-----+-----+

=== Energy Analysis ===
Fewer cycles = less CPU active time = less energy consumed.
Less stack = lower memory power consumption.

n=8 : Fewer cycles -> Iterative | Less memory -> Iterative
n=16: Fewer cycles -> Iterative | Less memory -> Iterative
n=32: Fewer cycles -> Iterative | Less memory -> Iterative

Conclusion: Iterative FFT is preferred for battery-powered devices.
- Fixed stack depth avoids recursive call overhead.
- No repeated function-call prologue/epilogue overhead.
- Predictable memory access pattern -> better cache utilization.

```

Figure 7: QEMU terminal: performance benchmark results showing cycle counts and stack usage

N	Iter Cycles	Rec Cycles	Iter Stack (B)	Rec Stack (B)
8	52,327 [†]	59,034 [†]	176	288
16	8,316	8,861	176	352
32	14,608	16,845	176	416

Table 1: Cycle counts and stack memory usage for iterative vs recursive FFT on QEMU.

[†]The n=8 cycle counts are inflated due to QEMU cold-start effects (instruction cache warming on first execution). The n=16 and n=32 measurements, taken after the cache is warm, are representative and follow the expected $O(N \log N)$ trend correctly (n=16 uses $\approx 8K$ cycles, n=32 uses $\approx 15K$ cycles, roughly double as expected).

7.2 Analysis

1. Which implementation uses fewer clock cycles?

The iterative FFT uses fewer cycles for all tested sizes ($n=8, 16, 32$). The recursive implementation has additional overhead from repeated function call prologue and epilogue at each recursion level, which adds up to $\log_2(n)$ extra frame saves/restores per FFT call.

2. Which uses less memory?

The iterative FFT uses a *fixed* 176 bytes of stack regardless of input size. The recursive FFT grows with input: 288 bytes for $n=8$, 352 for $n=16$, and 416 for $n=32$, because each recursive call adds a 64-byte stack frame and there are $\log_2(n)$ levels of recursion.

3. Which is expected to consume less energy and why?

- **Fewer cycles $\rightarrow$ less CPU active time $\rightarrow$ less dynamic power.** The iterative version consistently executes fewer cycles, meaning the CPU spends less time in an active (power-hungry) state.
- **Less memory $\rightarrow$ lower memory power consumption.** The iterative version's fixed stack footprint means no additional cache pressure or DRAM accesses from growing call frames.
- **Predictable access pattern.** Iterative loops access memory in a regular, sequential pattern, which is more cache-friendly than recursive calls that scatter stack frames across memory.
- The cycle count difference has a **greater impact** than stack size for energy, since CPU active power typically dominates SRAM leakage power.

4. Battery-powered device recommendation:

The **iterative FFT** would drain the battery more slowly. Its fixed stack usage creates no memory pressure at any input size, and eliminating function-call overhead entirely makes it the clear choice for continuous real-time processing on embedded or IoT devices.

8 Challenges and Solutions

8.1 RISC-V Floating Point Calling Convention

RISC-V uses separate floating-point registers (fa0–fa7, ft0–ft11, fs0–fs11). When calling `my_sin` from `generate_twiddle_factors`, the angle value in `fa0` was being overwritten by the return value. **Solution:** Spill the angle to the stack (`fsd/fld`) before the call and reload it afterward.

8.2 Twiddle Factor Indexing in Recursive FFT

The recursive butterfly operates on sub-arrays of varying sizes. The correct twiddle index for position j at a sub-problem of size n is $j \times (N_{\text{total}}/n)$. **Solution:** Pass `total_n` as a separate sixth parameter through all recursive calls to compute the index correctly at every depth.

8.3 GitHub Authentication

GitHub no longer accepts account passwords for `git push`. **Solution:** Generated a Personal Access Token (PAT) with `repo` scope from GitHub Settings → Developer Settings → Tokens (classic), and used it as the password when prompted.

8.4 Milestone 1 Edge Detection Bug

The M1 edge detection produced thick blobs instead of thin edges. Two bugs were found: (1) `create_highpass_filter` was called twice, once manually and once inside `edge_detection` - doubling the filter effect; (2) the cutoff frequency of `1.5f` was far too aggressive. **Solution:** Removed the manual call and set cutoff to `30.0f`, producing correct edge output.